

LLM Firm Lab — User Guide

A plain-English walkthrough for researchers who want to run simulations and explore the data. No coding experience required to follow along; you will be running Python commands but everything you need to type is spelled out.

What is this?

LLM Firm Lab is a simulated pharmaceutical industry where AI models play the roles that real humans would play in a real company — CEO, CFO, board member, auditor, sell-side analyst, SEC investigator, activist shareholder, banker. Every quarter, each AI reads the latest information available to its role and makes a decision: set a price, approve a loan, publish an analyst note, investigate a firm for accounting fraud.

The simulation tracks every decision and every dollar. At the end of a run, you get 21 research datasets in the exact format that real academic papers use (Compustat, ExecuComp, Audit Analytics, I/B/E/S, and more), plus a full audit trail of who decided what and why.

You use it to ask research questions like:

- Do CEOs get fired more often when the stock is doing badly?
- Does the SEC catch accounting fraud before an auditor does?
- Do interlocking directorships (directors who sit on multiple boards)

help firms forecast their rivals better?

- How do activist shareholder campaigns affect M&A outcomes?

You design an experiment, run the simulation, and analyze the output data.

Before you start — one-time setup

This section is a one-time chore. Once done, you never need to repeat it.

1. Install Python and dependencies

You need Python 3.10 or later. If you don't have it, download from [python.org](https://www.python.org/downloads/).

Open a terminal (PowerShell on Windows, Terminal on Mac/Linux), navigate to the project folder, then install the project's dependencies:

```
pip install -r requirements.txt
```

Wait for it to finish. You'll see a list of packages installing.

2. Get an OpenRouter API key

OpenRouter is a service that gives you access to many AI models (Claude, GPT, Qwen, Mistral, etc.) with one single key. It's pay-as-you-go and typical costs are a few cents per simulation run.

- Go to openrouter.ai.
- Create an account, add a small amount of credit (say \$10 to start —

it will last you a long time).

- Go to the Keys page and create a key. It looks like sk-or-v1-....

3. Save your key

Create a file named `.env` in the project folder. Copy your API key into it like this:

```
OPENROUTER_API_KEY=sk-or-v1-paste-your-key-here
```

Save the file. This file is private — don't share it or commit it to version control.

4. Verify the install

Run this command in the terminal (still in the project folder):

```
python -m pytest tests/ -q
```

You should see something like 285 passed in 3 seconds. If yes, you're set up. If not, see the Troubleshooting section at the end.

Your first run — the fast, free one

Before spending any money on API calls, try a "mock mode" run. Mock mode uses simple deterministic rules instead of real AI, so it's instant and free. It's a great way to check that everything works.

Run the smoke test

```
python -m src run --config config/test_stage12_mock.yaml --mock
```

You'll see quarter-by-quarter output scroll by for about 10 seconds. At the end you'll see something like:

```
=== RUN SCORECARD: run_1776789110 ===

FIRM PERFORMANCE:
  firm_0: NPV=$+182.8M | IRR=+45.9% ann. ...
  firm_1: NPV=$+201.4M | IRR=+49.6% ann. ...
  firm_2: NPV=$+221.8M | IRR=+53.6% ann. ...

DEBT PERFORMANCE:
  Loaned: $0.0M | Recovered: $0.0M | Loss rate: 0.0%

EQUITY PRICING ACCURACY:
  Mean error: $0.89/sh | MAPE: 2.2%

Outputs written: outputs/run_XXXXXX/
```

What just happened: you ran 8 quarters of simulation for 3 firms. Because it was "mock mode", the firms made predetermined decisions rather than having AIs think for them. But all the accounting, market dynamics, and data output worked normally.

Look at the outputs

Navigate to the `outputs/` folder, open the newest `run_XXXXXX/` subfolder. You'll find:

- `scorecard.txt` — firm performance summary
- `summary.txt` — one-paragraph run description
- `compustat_q.csv` — the main research dataset (one row per firm per quarter; opens in Excel)
- `firms/firm_0/`, `firms/firm_1/`, ... — per-firm narrative outputs

(board meetings, R&D reports, annual reports)

- 20+ other CSVs covering every aspect of the simulation

Your first real run — with AI

Now let's run with real AIs making decisions. This will use your API key and cost a small amount (typically 3-10 cents per run).

Start small

```
python -u -m src run --config config/validation_v15_theta.yaml
```

Settings: 3 firms, 4 quarters (1 fiscal year), all features on. Expect roughly 60-75 minutes wall-clock time and about \$0.03-0.05 USD.

While it runs, you'll see messages like:

```
firm_0: mistralai/mistral-small-24b-instruct-2501 [openrouter]
firm_1: qwen/qwen3-235b-a22b-2507 [openrouter]
...
Q1 2031: Rev=$15.0M | Firms=3 | Gen=G1,G1,G1 | 855.3s
Q2 2031: Rev=$15.0M | Firms=3 | Gen=G1,G1,G1 | 1342.4s
...
```

Each quarter takes a few minutes because several AI calls happen in sequence. You can safely walk away and come back later.

When it finishes

You get the same outputs as the mock run, but now the board minutes, audit reports, and analyst notes are all written by AI. Open `outputs/run_XXXXXX/firms/firm_0/board_minutes_Q2.md` in a text editor — you'll see a realistic board discussion about the firm's strategy.

You'll also see a new file: `cost_summary.txt`. It tells you exactly how much the run cost:

```
=== LLM COST / TOKEN SUMMARY ===
Total calls: 104
Total input tokens: 164,208
Total output tokens: 102,124
Total tokens: 266,332
Wallclock: 4508.4s
Estimated cost: $0.0326 USD
```

Understanding the outputs

Every run produces a lot of data. Here's where to look for what.

The scorecard — did the firms do well?

`outputs/run_XXXXXX/scorecard.txt` summarizes firm performance from the perspective of the IPO shareholder. NPV is the net present value of all cash flows to shareholders; IRR is the annualized return. Positive NPV means the firm created value; negative means it destroyed value.

The narrative files — what did the AI think?

Each firm has its own folder with human-readable files:

- board_minutes_Q<N>.md — the board discussion for quarter N
- rd_report_Q<N>.md — R&D progress report
- annual_report_FY<YEAR>.md — 10-K-style annual report
- product_spec.md — what the firm's product is

These are written as prose and are a fun way to see the AI's reasoning.

The research datasets — the actual numbers

These are CSV files in outputs/run_XXXXXX/ that you can open in Excel or load in pandas. The main ones:

File	What it is
compustat_q.csv	Quarterly financials — revenue, cash, debt, earning
compustat_a.csv	Annual version of the same.
execucomp.csv	CEO compensation per year. Salary, bonus, stock awa
audit_analytics.csv	Annual audit opinions. Which auditor, clean or qual
analyst_forecasts.csv	Every analyst note — EPS forecast, target price, ra
restatements.csv	Every time a firm restated its earnings, and why.
ceo_turnover.csv	Firings, retirements, new-hire events.
debt_facilities.csv	Every loan the firms took out.
covenant_violations.csv	Every time a firm broke a loan covenant.
activist_campaigns.csv	Every activist shareholder demand.
insider_transactions.csv	CEO stock grants, sells, option exercises.
director_turnover.csv	Director retirements, appointments, departures.
annual_reports.csv	The full 10-K data for each firm each year.
crosswalk.csv	Maps every entity (firm, CEO, director, facility) t

There are more — see docs/datasets.md for the complete list.

The audit trail — why did X happen?

These files are less pretty but very useful for research:

- proposals.jsonl — every decision by every AI, with full reasoning.

Each line is a structured record. Searching this file tells you "why did firm_2 cut its R&D in Q3?" — you find the AI's reasoning.

- negotiations.jsonl — every multi-round negotiation (debt pricing, covenant waivers, audit fee haggles, activist campaigns, M&A auctions).

- bs_violations.jsonl — if this file is empty (which it should be on a clean run), your accounting is internally consistent.

- peer_observations.jsonl — each firm's noisy observation of its peers each quarter. Useful for research on the interlocking-director information-leak mechanism.

- cost_summary.txt + llm_calls.jsonl — exactly how much each AI role cost.

The dashboard — a visual overview

The easiest way to explore results without opening CSVs is the Streamlit dashboard. Run this:

```
python -m streamlit run app/dashboard.py
```

A browser window opens. You'll see a page with:

- Sidebar: pick which runs to include
- 16 tabs: time series, ratios, CEO compensation, turnover, debt

covenants, analyst forecasts, data integrity checks, earnings management heatmaps, firm comparisons, cross-run distributions, auditor timelines, proposals browser, negotiations, regressions, crosswalk, cost

Click around. Hover over charts for details. The dashboard works with both mock and live runs.

Features you can turn on and off

The simulation has about 30 feature toggles. Every feature defaults to "safe" (backward-compatible with older runs). To experiment with a specific feature, edit your config YAML file (for example config/validation_v15_theta.yaml) and change the toggle.

The major toggles and what they do

Toggle	What happens when ON
earnings_management_enabled	Firms can manipulate reported earnings. Creates the
sec_enabled	SEC monitors all firms, investigates suspicious pat
auditor_enabled	At year-end, one of 4 named auditors reviews each f
governance_enabled	At year-end, the board reviews the CEO, sets pay, c
sellside_analysts_enabled	3 sell-side analysts publish ratings and target pri
earnings_announcement_enabled	Firms issue quarterly earnings releases with manage
restatements_enabled	When manipulation is detected, prior periods are re
ma_enabled	Firms can bid to acquire rivals. Multi-round auctio
debt_covenants_enabled	Loans come with covenants; violations trigger waive
activist_investors_enabled	An activist fund scans the industry each quarter an
legal_reserves_enabled	Firms accrue reserves for litigation risk.
pension_enabled	Firms operate defined-benefit pension plans.
deferred_taxes_enabled	Book-tax differences create deferred tax assets/lia
stock_comp_enabled	CEOs get RSU and option grants with vesting schedul
bad_debt_enabled	Receivables get an allowance; some become uncollect
directors_enabled	A shared pool of directors populates boards, with i
director_lifecycle_enabled	Directors rotate out annually; defaulted-firm seats
three_llm_board_enabled	Board governance uses 3 separate AI voices (CEO-, C
noisy_signals_enabled	Firms see competitor prices/revenues with Gaussian
annual_reports_enabled	At year-end each firm writes a 10-K-style annual re
strategic_planning_enabled	Firms author forward 5-year strategic plans at Q0 +
pe_lifecycle_enabled	Firms start PRIVATE with 5% of scenario's founding_

To enable/disable a feature

Open the config file in a text editor. Find the line, change true to false or vice versa. Save and re-run.

Example — turn off M&A for a cleaner baseline:

```
ma_enabled: false
```

Scenarios — choose your industry

By default all firms start with identical conditions. Sometimes you want heterogeneity — firms with different starting cash, CEOs with different personalities, different R&D maturity.

Three pre-built scenarios:

- `biotech_early_stage` — high burn rate, \$200M+ cash reserves, no revenue yet
- `mature_industry` — established players, steady margins, modest growth
- `distressed` — leveraged firms in fragile cash positions

To use a scenario, add one line to your config:

```
scenario: biotech_early_stage
```

Cost and time estimates

All numbers below are measured on real runs using the OpenRouter pricing of late April 2026. Mix of models: mistral-small-24b, qwen3-235b, glm-4-32b, phi-4, gemini-2.0-flash, deepseek-v3.2, gemma-12b.

Config	Firms	Quarters	Time	Cost (USD)	Source
Mock smoke (no AI)	3	8	~10 sec	\$0.00	free
Small live (1 fis	3	4	75 min	\$0.033	measured (v15)
Standard live (1	5	4	78 min	\$0.047	measured (1y run)
Research run (2 f	5	8	~2.5 hours	~\$0.09	extrapolated (×2)
10-seed panel	5 × 10 seeds	8	~20+ hours	\$0.90-1.00	extrapolated (×10)
10-seed panel, pa	5 × 10	8	~5 hours	\$0.90-1.00	with batch_runner

How costs scale

- By firms: sub-linear in wall-clock (firm decisions run in parallel per quarter); roughly linear in cost. Going from 3 → 5 firms added 40% to cost but only 4% to wall-clock because of parallelization.
- By quarters: linear in both time and cost. A 2-year run costs roughly twice a 1-year run.
- By seeds: linear in total cost (each seed is an independent run). Wall-clock reduces if you run seeds in parallel via `batch_runner.py`.

When the 3-LLM board committee is on

Turning on `three_llm_board_enabled` replaces the 1-call governance review with 3 perspective calls (CEO/CFO/comp-committee) plus a synthesis — 4× the governance-phase cost. At 5 firms × 1 year that's roughly an extra \$0.01-0.02 USD, and adds ~5-10 minutes wall-clock (the 3 voices run in parallel, so it's the synthesis call that takes the extra time).

Real-world budgeting

- Exploring the simulation: a handful of \$0.03-0.05 runs totals under a dollar. Your OpenRouter account's \$10 opening credit covers dozens of experiments.
- A publishable paper: the 10-seed × 2-fyear panel at ~\$1 total gives you 2,000+ firm-quarters with full structured-action provenance — plenty for standard corporate-finance regressions.

- Longer horizons: 5-firm × 4-year runs run \$0.18-0.20 each;

budget accordingly if you need multi-year dynamics.

All runs write `cost_summary.txt` to the run folder with a per-model and per-agent-role breakdown, so you always know exactly where the money went.

Running batches of experiments

For real research you usually want many runs with different seeds to measure statistical power.

```
python scripts/batch_runner.py --config config/validation_v15_theta.yaml --seeds 42,43,44,45,46
```

This runs 5 simulations sequentially with different random seeds, writing each into a separate output folder. You can compare across runs using the dashboard's "Cross-run dist" tab or the meta-analysis script:

```
python scripts/meta_analysis.py
```

Running regressions — research in one command

Twelve built-in regression specifications produce standard corporate-finance paper tables:

- Pay-performance sensitivity (Jensen-Murphy)
- Leverage determinants (Rajan-Zingales)
- Covenant violations → default hazard
- CEO forced turnover
- Earnings-management detection
- Analyst forecast bias
- Event study: SEC actions and stock returns
- Event study: restatement announcements and returns
- Event study: CEO turnover and returns
- Matched-firm pricing study
- Disclosure tone → next-year returns
- Interlocking-director info leak → observation accuracy

Run all 12 against your latest run:

```
python scripts/baseline_regressions.py --runs run_XXXXXX
```

Text summaries are written to `outputs/regressions/*.txt` (one per spec). Open them in any text editor.

Going deeper — exporting to Excel or Python

Excel

Every `compustat_q.csv`, `execucomp.csv`, etc. opens directly in Excel. Column names follow WRDS conventions where an analog exists (`saleq` = sales quarterly, `atq` = total assets quarterly, `ceqq` = common equity quarterly). See `docs/datasets.md` for the complete column dictionary.

Python / pandas

```
import pandas as pd
```

```
df = pd.read_csv("outputs/run_XXXXXX/compustat_q.csv")
# Filter to one firm
firm0 = df[df["firm_id"] == "firm_0"]
# Plot revenue over time
firm0.plot(x="datadate", y="saleq")
```

The cross-run accumulated database is in data/:

```
all_runs = pd.read_csv("data/compustat_all.csv")
# Now you have every quarter of every run stacked
```

Troubleshooting

"No module named 'src'" — You're not in the project folder. Run `cd` to get there first.

"OpenRouter API key not found" — Your `.env` file is missing or in the wrong location. It should be in the project root (same folder as `README.md`).

"Rate limited" — OpenRouter is throttling you. The simulation will automatically wait and retry. If it happens too much, reduce `parallel_firm_decisions` to `false` in your config.

The run took way longer than estimated — Qwen and GLM-4 occasionally take 30+ seconds per call when OpenRouter routes to a slow provider. A single slow quarter can add 10-20 minutes. Usually self-resolves.

"Bad JSON response" warnings — Some AIs occasionally return malformed JSON. The simulation has automatic retry built in (5 attempts with exponential backoff). You can ignore these unless the run actually crashes.

Tests fail after I changed something — Revert your change. Tests are the truth; if they fail, your change broke something.

The dashboard is blank — Run a simulation first. The dashboard only shows runs that exist in `outputs/`.

Where to learn more

Inside the project folder:

- `CHANGELOG.md` — history of every feature added, wave by wave
- `docs/ROADMAP.md` — what's done, what's planned
- `docs/principles_review.md` — the 20 design principles and how each

is verified in the code

- `docs/architecture.md` — how all the pieces fit together
- `docs/datasets.md` — every output CSV column explained
- `docs/SIMULATION_SUMMARY.md` — one-page summary of the project
- `CODEX_AUDIT_FILLED.md` — independent audit report (Codex verified

28/33 specific claims about the code)

For help with a specific module or question, open an issue (if this is a git-managed project) or ask someone who writes code to look at the file in `src/`.

Summary — the shortest version

- Install Python, dependencies, and an OpenRouter API key (one time).

- Run `python -m src run --config config/test_stage12_mock.yaml --mock` to verify everything works (free, 10 seconds).
- Run `python -u -m src run --config config/validation_v15_theta.yaml` for a real AI-driven simulation (~1 hour, ~\$0.03).
- Look at `outputs/run_XXXXXX/scorecard.txt` for the result.
- Run `python -m streamlit run app/dashboard.py` to browse all data visually.
- Change toggles in the config YAML to ask new research questions.

Everything else in this document is elaboration on these six steps.